BTS CIEL IR — CARNUS ENSEIGNEMENT SUPÉRIEUR, RODEZ

Mini-Projet 2 — Contrôle d'accès, authentification et réservation de la salle 215

Rapport Commun — Octobre 2026 Spécialité IR (Informatique et Réseaux)

Composition du groupe de projet :

Tâche 1 (RFID) :
Youssef SAIDI
Alban PUECH

Tâche 2 (Site Web) :
Tony GIANSILI
Nielsen NABTI

Tâche 3 (Serveur & API) :
Grégoire FEROUELLE

1. CONTEXTE, PROBLÉMATIQUE ET OBJECTIFS

1.1 Contexte

La salle 215 est utilisée par plusieurs personnes à des horaires différents. Aujourd'hui, rien ne permet de savoir qui peut y entrer, à quel moment, ni de garder une trace des passages. Le mini-projet 2 consiste à concevoir un système complet qui contrôle l'accès à cette salle et permet de la réserver.

1.2 Problématique

Comment autoriser l'entrée dans la salle 215 uniquement aux bonnes personnes, au bon moment, tout en gardant une trace de chaque tentative d'accès ?

1.3 Objectifs

- Lire un badge RFID à l'entrée de la salle et décider en temps réel si l'accès est autorisé ;
- Créer un compte, se connecter et réserver la salle depuis un site web dédié ;
- Lier l'accès au calendrier : un client n'entre que pendant le créneau qu'il a préalablement réservé ;
- Enregistrer chaque tentative d'accès ainsi que chaque action importante réalisée sur le site ;
- Assurer un fonctionnement sécurisé sur un serveur Linux commun aux trois tâches.

2. CAHIER DES CHARGES ET CONTRAINTES

2.1 Exigences fonctionnelles

EXIGENCE FONCTIONNELLE	TÂCHE PRINCIPALE
Lecture d'un badge RFID et signalisation lumineuse (LED verte / rouge)	Tâche 1
Autorisation ou refus de l'accès selon le rôle et la réservation active	Tâche 3
Journalisation systématique de chaque tentative d'accès (horodatage, UID)	Tâche 3
Inscription, connexion et gestion des rôles (Administrateur / Client)	Tâche 2
Consultation des disponibilités et création de réservations de créneaux	Tâche 2
Vérification dynamique des conflits et chevauchements de créneaux	Tâche 2
Génération d'un QR code iCalendar associé à chaque réservation	Tâche 2

2.3 Contraintes techniques et d'organisation

- **Hébergement :** serveur sous Linux (Raspberry Pi OS Lite Debian 13), installé dans la salle et allumé en permanence ;
- **Réseau :** fonctionnement sur le sous-réseau de la salle (`172.16.10.0/24`) avec une IP fixe validée par l'enseignant ;
- **Sécurité des dépôts :** aucun mot de passe ni clé d'API dans le dépôt public GitHub ;
- **Chiffrement :** échanges sensibles chiffrés lorsque c'est possible (HTTPS) ;
- **Organisation :** travail réparti en trois tâches développées en parallèle par l'équipe.

3. ORGANISATION DU GROUPE ET RÉPARTITION DES TÂCHES

3.1 Répartition des rôles

TÂCHE	RESPONSABLES	PÉRIMÈTRE & MATÉRIEL
Tâche 1 — Contrôle RFID	Youssef SAIDI Alban PUECH	ESP32, lecteur RFID MFRC522 (13.56 MHz), LED verte/rouge, envoi UID via API.
Tâche 2 — Web & Réservations	Tony GIANSILI Nielsen NABTI	Pages PHP, authentification, gestion des rôles, réservations, QR codes.
Tâche 3 — Serveur & API	Grégoire FEROUELLE	Raspberry Pi, MariaDB, Apache, API REST, sécurité UFW, intégration globale.

** Note : La tâche 3 était initialement prévue pour un binôme. Réalisée par une seule personne, la priorité a été donnée aux éléments structurants indispensables aux autres tâches : la base de données et les contrats d'échange de l'API.*

3.2 Méthode de travail et coordination GitHub

Le groupe a structuré son travail autour d'un dépôt GitHub commun selon une démarche collaborative rigoureuse :

- Une branche `main` centrale et une branche par tâche, complétées par des sous-branches de fonctionnalités ;
- Suivi du projet par *Issues* GitHub assignées individuellement et intégrées dans un Kanban ;
- Validation des modifications par *Pull Requests* avant fusion.

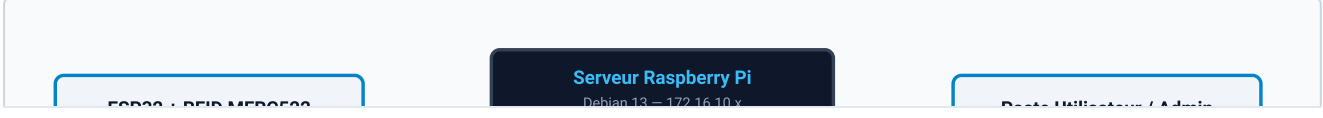
La synchronisation de l'équipe s'est articulée en trois étapes majeures :

1. **Contrat d'interface précoce :** fixation dès le début de la structure MariaDB et du format JSON de l'API par la Tâche 3, permettant un développement autonome de l'ESP32 (Tâche 1) et du site web (Tâche 2).
2. **Développement parallèle :** simulation locale du site web par la Tâche 2 sous XAMPP à l'aide des scripts SQL fournis.
3. **Intégration serveur :** déploiement sur le Raspberry Pi et recette globale en conditions réelles.

4. ARCHITECTURE GÉNÉRALE ET INTERACTIONS

4.1 Architecture système et réseau

Le serveur (Tâche 3) occupe la position centrale. Les composants satellites y accèdent de manière cloisonnée :



4.1 Matrice d'accès aux données

COMPOSANT	MODE D'ACCÈS	JUSTIFICATION & SÉCURITÉ
ESP32 (Tâche 1)	Requetes HTTP via l'API `/api/rfid.php`	Appareil embarqué distant : l'accès direct SQL est proscrit pour éviter d'exposer la base.
Site Web (Tâche 2)	Connexion SQL directe en `localhost`	Le site est colocalisé sur le même serveur Raspberry Pi que la base MariaDB.

Règle d'isolation : La base MariaDB est totalement inaccessible depuis le réseau extérieur. Seuls l'API locale et le site web ont l'autorisation de lire et d'écrire dans la base.

4.2 SCÉNARIO D'ACCÈS PAR BADGE RFID

Lorsqu'un utilisateur présente son badge devant le lecteur de la salle 215, le traitement s'effectue en temps réel selon la séquence suivante :



Figure 2 : Diagramme de séquence détaillé du passage d'un badge RFID

1. L'utilisateur présente son badge devant le lecteur MFRC522.
2. L'ESP32 lit l'UID du badge et l'envoie à l'API du serveur en JSON, accompagné de la clé d'API.
3. Le serveur valide la clé, authentifie le propriétaire, consulte son rôle et vérifie l'existence d'une réservation active.
4. Le serveur inscrit la tentative dans la table `acces`, puis retourne un résultat (`autorise`, `refuse`, `inconnu`, `erreur`).
5. L'ESP32 interprète le retour et pilote le témoin lumineux (LED verte = accès accordé / LED rouge = accès refusé).

4.3 SCÉNARIO DE RÉSERVATION

1. L'utilisateur accède au site web sécurisé et s'authentifie.
2. Il consulte la grille des disponibilités et sélectionne le créneau souhaité pour la salle 215.
3. Le site vérifie instantanément l'absence de chevauchement avec une réservation existante.
4. La réservation est validée en base et un QR code au format iCalendar est généré.
5. Pendant ce créneau exact, le badge de l'utilisateur est déverrouillé par le serveur.

Note d'articulation : Le lien entre le site web et l'accès physique s'effectue au niveau de l'utilisateur. Le QR code fournit une fiche calendaire à l'utilisateur pour son smartphone, mais seul le badge RFID constitue le moyen de déverrouillage physique.

4.4 DONNÉES COMMUNES — BASE DE DONNÉES MP2_SALLE215

La base MariaDB regroupe sept tables partagées entre l'API REST et le site web :

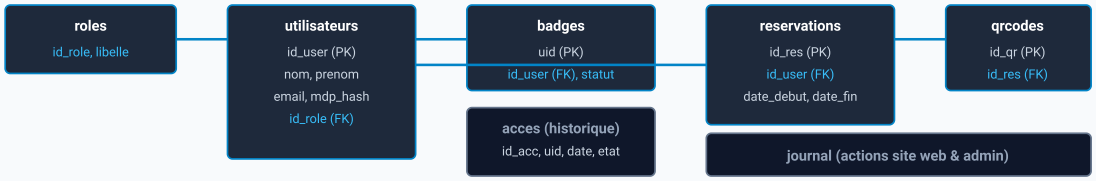


Figure 3 : Schéma relationnel de la base de données mp2_salle215

Roles & Utilisateurs

Gestion des comptes (admin/client), identifiants d'accès et hachage sécurisé des mots de passe.

Badges & Réservations

Association des identifiants physiques RFID et suivi des créneaux horaires réservés.

Qrcodes

Stockage du jeton iCalendar lié aux événements d'agenda sur smartphone.

Acces & Journal

Traçabilité double : tentatives de passages RFID et logs d'administration du site.

5. MATÉRIELS, LOGICIELS ET TECHNOLOGIES

Les choix technologiques ont été motivés par les contraintes industrielles et économiques du projet :

Matériel RFID	Serveur Hôte	Stack Web
ESP32 + Lecteur MFRC522	Raspberry Pi OS Lite	Apache 2.4, PHP 8.4
Wi-Fi & coût réduit	Debian 13 sobrieté	MariaDB SQL

5. SYNTHÈSE COMPARATIVE DES TECHNOLOGIES RETENUES

DOMAINE	CHOIX TECHNIQUE	JUSTIFICATION TECHNIQUE
Lecteur de badge	ESP32 + MFRC522 (13,56 MHz)	Microcontrôleur Wi-Fi intégré, faible coût, bibliothèques C++ matures.
Serveur système	Raspberry Pi (Debian 13 Lite)	Fonctionnement continu en salle, faible consommation électrique.
Serveur Web	Apache + PHP 8.4	Uniformité de langage pour l'interface web et les endpoints d'API REST.
Base de données	MariaDB 10.11	Performance, robustesse et compatibilité directe avec l'environnement XAMPP.
Échanges ESP32	HTTP + JSON + Clé API	Mise en œuvre légère et très rapide sur microcontrôleur embarqué.
Échanges Web	HTTPS (Certificat Auto-signé)	Garantie du chiffrement des identifiants et des jetons de session web.
Sécurité réseau	Pare-feu UFW, SSH par clés	Filtrage strict des flux : seuls les ports 22, 80 et 443 sont ouverts.
QR Code	Format standard iCalendar (vCalendar)	Interopérabilité native avec les applications agenda sur smartphone.
Outillage groupe	GitHub, VS Code, Postman, Wireshark	Gestion des sources, recette des endpoints API et analyse de trames réseau.

6. FONCTIONNALITÉS RÉALISÉES ET RÉSULTATS

6.1 Bilan des fonctionnalités

✔ **Lecture RFID & LED :** MFRC522 opérationnel avec signalisation lumineuse.

✔ **Décision d'accès :** Contrôle des rôles et des créneaux de réservation.

✔ **Historique de passage :** Journalisation continue des tentatives dans la base.

✔ **Gestion des comptes :** Authentification, inscription, rôles Client / Admin.

✔ **Administration :** Panneau de gestion des utilisateurs et des droits.

✔ **Anti-chevauchement :** Détection dynamique des conflits d'horaires.

✔ **Module QR Code :** Exportation de la réservation vers l'agenda mobile.

✔ **Sécurité renforcée :** Pare-feu UFW, HTTPS, requêtes préparées PDO.

6.3 Mesures de performance serveur

7 ms

Temps de réponse moyen de l'API REST (mesuré sur le serveur)

270 Mo / 905 Mo

Empreinte mémoire RAM au repos (12 % d'occupation disque)

6.2 Test final d'intégration (Recette du 15 Octobre 2026)

Le système complet a été soumis à une campagne d'essais en conditions réelles d'exploitation :

HEURE	ACTION RÉALISÉE	RÉSULTAT & CONSTAT
10:53	Badge d'une cliente présenté sans aucune réservation active	REFUSÉ LED Rouge allumée
11:03	Réservation sur le site web pour le créneau 11:03 – 11:20	SUCCÈS QR code généré
11:05	Tentative de réservation chevauchant le créneau précédent	REFUSÉ Message d'erreur conflit
11:06	Présentation du même badge client durant son créneau	AUTORISÉ LED Verte allumée
11:07	Annulation de la réservation puis présentation du badge	REFUSÉ Logué au journal

✔ **Validation d'intégration : La prise en compte des actions du site web sur le comportement du lecteur physique est immédiate.**

6.4 Limites identifiées

- **Trame HTTP de l'ESP32 :** La clé d'API circule en clair sur le réseau local (vérifié via Wireshark).
- **Certificat auto-signé :** Avertissement de sécurité affiché par le navigateur web à la connexion HTTPS.
- **Double lecture RFID :** Rebond matériel enregistrant parfois deux passages dans la même seconde.
- **Génération du QR Code :** Dépendance à un service en ligne externe sans secours local en cas de coupure Internet.

7. DIFFICULTÉS, SOLUTIONS ET BILAN

7.1 synthèses des difficultés rencontrées

PROBLÈME / SYMPTÔME	CAUSE RACINE & SOLUTION APPORTÉE
Incapacité de l'ESP32 à joindre l'API	Adresses d'API et de test inversées dans le C++. Correction après diagnostic `tcpdump`.
Échec du test réseau (Ping)	Redirection HTTP 302 vers la page de login. Création d'une route dédiée `/api/ping.php`.
Différence de configuration MariaDB	Identifiants distincts entre XAMPP et le Pi. Utilisation d'un fichier de config isolé (hors Git).
Masquage d'adresse de l'ESP32	NAT effectué par le point d'accès Wi-Fi. Identifier sans impact sur les réponses de l'API.

7.2 Bilan synthétique

Le système répond pleinement aux exigences du cahier des charges. La définition précoce des contrats d'interface (API JSON et structure MariaDB) a été le facteur clé du succès, permettant un développement en parallèle efficace des trois tâches.

7.3 Perspectives d'évolution

- • Migration de l'ESP32 vers du HTTPS (TLS)
- • Filtrage anti-rebond (délai de 2s post-lecture)
- • Génération locale des images QR Code
- • Sauvegardes automatisées nightly de la BDD